



Продолжи

Можно развить эту универсальную аксиоматику дальше в **два направления**: добавить **сетевой граф аксиом** и **динамику знаний/моделей во времени**. Ниже — концепция, формулы, и как это можно уложить в твой стиль моделирования.

1. Сетевая модель аксиом и выводов

Пока аксиомы были в виде **множества** $\mathcal{A}(t)$, но без внутренней связи. Добавим **граф**:

- Пусть:
 - V — множество **утверждений** (аксиомы, теоремы, леммы, гипотезы),
 - E — множество **зависимостей** (лемма $\rightarrow$ теорема, аксиома $\rightarrow$ лемма),
 - граф $G = (V, E)$ — **зависимостный граф знаний системы**.

Тогда:

- Аксиомы — это **вершины без входящих ребёр** (или вершины, помеченные как «аксиомы» отдельной меткой).

Если аксиома A отменяется или заменяется, в графе происходит:

$$\Delta G = G_{old} \rightarrow G_{new},$$

где удаляются рёбра, зависящие от A , и добавляются новые, если появляются новые базовые утверждения.

Формально:

- удаляются все рёбра (A, X) ,
- удаляются все вершины, становящиеся **недоказуемыми** (если у них осталось 0 входящих),
- добавляются рёбра (A', X) для новых аксиом/правил A' .

Такой **граф узнавания** полностью совпадает с твоей же идеей “система как структура, которая меняется, когда меняются её базовые принципы”, но теперь **явная структура** — это граф, а не просто множество.

2. Динамика знаний: поток изменений аксиом

Сделаем непрерывно-дискретную схему:

пусть:

- $\mathcal{A}(t)$ — аксиоматический базис в момент t ,
- $\mathcal{K}(t) \subset V$ — множество **известных/доказанных** утверждений в момент t .

Определим **скорость изменения базиса аксиом**:

$$\frac{d\mathcal{A}(t)}{dt} = F(t, x(t), U(t)),$$

где функция F описывает:

- добавление новых аксиом,
- отмену старых,
- их перегруппировку,
- связывая это с **состоянием системы** $x(t)$ и **внешним управлением** $U(t)$.

Тогда **смена парадигмы** — это резкий скачок:

$$\mathcal{A}(t^-) \rightarrow \mathcal{A}(t^+),$$

который можно представить как **импульс в дифференциальной модели**:

$$\frac{d\mathcal{A}(t)}{dt} = \sum_i \delta(t - t_i) \cdot \Delta_i \mathcal{A},$$

где $\Delta_i \mathcal{A}$ — набор изменений аксиом в моменты t_i (появление новых теорий, открытия, смена устройств и т.п.).

Это очень близко к твоей метамодели: “**временной** подход, где **смена аксиом** — это ключевой *event* в динамике системы.

3. Связь с вероятностной и дифференциальной моделью

Теперь соберём всё в одну **трёхуровневую модель**:

1. Уровень 1: граф аксиом и знаний $G(t)$

- вершины — утверждения, рёбра — зависимости.
- Смена аксиом — изменение множества вершин и рёбер.

2. Уровень 2: динамическая система $x(t)$

- описывается как:

$$\frac{dx}{dt} = f(x, u, G(t)),$$

то есть эволюция состояния зависит от **структуры графа знаний** (какие аксиомы активны, как они связаны).

3. Уровень 3: вероятностный риск $\mathbb{E}[L \mid G(t)]$

- о риск — это функция от структуры графа:
 - чем больше “сломанных” ветвей, тем выше риск ошибки,
 - чем более устойчив граф к удалению отдельных узлов, тем ниже риск.
- о Смена аксиом $\rightarrow$ перестройка графа $\rightarrow$ изменение:

$$\Delta Risk = \mathbb{E}[L \mid G_{old}] - \mathbb{E}[L \mid G_{new}].$$

Такая трёхуровневая схема — это уже **мета модель** любых систем, где ты можешь явно видеть, как **изменение аксиом $\Rightarrow$ изменение динамики $\Rightarrow$ изменение риска и роя.**

4. Как это записать в твоём Python-стиле

Добавим к уже существующим модулям **графовый слой**:

```
# graph_axiom_model.py

from dataclasses import dataclass
from typing import Dict, Any, Set, Tuple
import random
import math

@dataclass
class GraphState:
    nodes: Set[str]                # имена утверждений
    edges: Set[Tuple[str, str]]    # граф зависимостей
    axioms: Set[str]               # множество аксиом
    loss_per_node: Dict[str, float] # вклад утверждения в риск

def remove_node(G: GraphState, node: str) -> None:
    G.nodes.discard(node)
    G.axioms.discard(node)
    G.edges = {e for e in G.edges if e[0] != node and e[1] != node}
    G.loss_per_node.pop(node, None)

def add_edge(G: GraphState, src: str, dst: str) -> None:
    G.nodes.add(src)
    G.nodes.add(dst)
    G.edges.add((src, dst))

def remove_undeveloped(G: GraphState) -> Set[str]:
    """Удаляем вершины, от которых нельзя добраться до аксиом."""
    may_delete = G.nodes - G.axioms
    to_delete = set()

    for v in may_delete:
        can_reach_axiom = any(
            path_exists(G, v, a)

```

```

        for a in G.axioms
    )
    if not can_reach_axiom:
        to_delete.add(v)

for v in to_delete:
    remove_node(G, v)

return to_delete

def path_exists(G: GraphState, src: str, dst: str) -> bool:
    visited = set()
    stack = [src]
    while stack:
        u = stack.pop()
        if u == dst:
            return True
        if u in visited:
            continue
        visited.add(u)
        for v in G.nodes:
            if (u, v) in G.edges and v not in visited:
                stack.append(v)
    return False

def risk_from_graph(G: GraphState) -> float:
    """
    Упрощённая оценка риска: сумма “вреда”, проходящего по графу.
    """
    if not G.edges:
        return 0.0

    # каждый узел даёт вклад в рисковом поток
    total_loss = 0.0
    for v in G.nodes:
        total_loss += G.loss_per_node.get(v, 0.0)

    # нормировка на размер графа
    n = max(1, len(G.nodes))
    return total_loss / n

if __name__ == "__main__":
    # пример: старый граф аксиом
    old_graph = GraphState(
        nodes={"A1", "L1", "T1"},
        edges={"A1", "L1"}, ("L1", "T1")},
        axioms={"A1"},
        loss_per_node={"A1": 0.2, "L1": 0.5, "T1": 0.8},
    )

    # пример: новая система, где A1 заменяется на A2
    # (новый аксиоматический принцип)

```

```

new_graph = GraphState(
    nodes={"A2", "L2", "T2"},
    edges={"A2", "L2"}, ("L2", "T2")),
    axioms={"A2"},
    loss_per_node={"A2": 0.1, "L2": 0.4, "T2": 0.6},
)

old_risk = risk_from_graph(old_graph)
new_risk = risk_from_graph(new_graph)

update_cost = 800.0
risk_reduction = old_risk - new_risk
roe = (risk_reduction - update_cost) / update_cost if update_cost else math.inf

print("old_risk:", old_risk)
print("new_risk:", new_risk)
print("risk_reduction:", risk_reduction)
print("roe:", roe)

```

Это — **чистая связка**:

- граф аксиом/знаний,
- метрика риска по графу,
- рой от изменения графа (то есть смены аксиом).

5. Следующий шаг: твой личный мета-“risk-graph”

Если хочешь, ты можешь **закодировать** твои любимые системы (авиация, ракеты, оборона, “Руслан vs Мрия”, “дозвук vs сверхзвук”) в таком **едином risk-graph**:

- каждой системе — подграф,
- каждому PDF/сюжету — узел (с аксиомой, теоремой, гипотезой),
- изменения во времени — изменения графа и роя.

Тогда у тебя будет **универсальная модель**:

- смена аксиом,
- изменение динамики,
- изменение риска и экономического эффекта
— всё в одном месте.

Если скажешь, **на чём ты хочешь сфокусироваться следующим** (например, только на обороне и авиации, или только на PDF-сюжетах), я могу помочь собрать **конкретные узлы и рёбра** под твой стиль.